

**BooN: Boolean Network Analysis Software**

Journal:	<i>Bioinformatics Advances</i>
Manuscript ID	BIOADV-2024-278.R1
Manuscript Type:	Application Note
Date Submitted by the Author:	n/a
Complete List of Authors:	Delaplace, Franck; Paris-Saclay University, Computer Science; University of Evry-Val-d'Essonne, Computer Science
Portal Keywords:	Networks, Graph theory, Mathematical modeling, Systems biology, Biological networks

SCHOLARONE™
Manuscripts

1

2

3

4

5

6

7

8

9

10

11

12

13

14

15

16

17

18

19

20

21

22

23

24

25

26

27

28

29

30

31

32

33

34

35

36

37

38

39

40

41

42

43

44

45

46

47

48

49

50

51

52

53

54

55

56

57

58

59

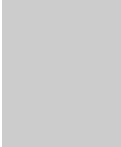
60

Bioinformatics, 2024, 1–5

doi: DOI HERE

Advance Access Publication Date: Day Month Year

Paper



BooN: Boolean Network Analysis Software

Franck Delaplace^{1,*}

¹IBISC, Paris-Saclay University, Evry, France

*Corresponding author. franck.delaplace@univ-evry.fr

FOR PUBLISHER ONLY Received on Date Month Year; revised on Date Month Year; accepted on Date Month Year

Abstract

In this paper, we present an open source software, called **BooN**, which has been developed for the purpose of Boolean network analysis in Python. The distribution includes two main packages: one comprising a set of methods applied to the Boolean network object and the other for the graphical interface. The software encompasses all the requisite features for Boolean network analysis, including graphical network design, interaction graph drawing, model of dynamics, symbolic computation of stable states, and controllability analysis. **BooN** can be freely accessed on GitHub [1].

Key words: Boolean Network, Controllability, Software

1. Introduction

Boolean Network Analysis applied to biological systems studies was pioneered by Stuart Kaufman [2, 3] and René Thomas [4] in 70s. The goal was to develop a formal modeling framework for studying the dynamics of genetic networks, considering the available data for their analysis.

Currently, such a modeling framework can be considered as a Gold standard for biological system study as evidenced by the significant number of publications on biological network and system modeling (Figure 1). Indeed, these publications have mostly supplanted other modeling frameworks regarding quantity.

One of the most significant challenges for this modeling framework concerns the development of software platforms facilitating their use by a large number of users with a diverse range of skill sets. Such platforms should offer a comprehensive range of functionality and a user-friendly graphical interface.

This article introduces **BooN** a Python-based platform that provides a comprehensive set of functionalities for Boolean network analysis and a user-friendly graphical interface for Boolean network manipulation.

Section 2 recalls the main features of a Boolean network analysis including equilibria computation and controllability analysis. Section 3 details the different functionalities of **BooN**. Section 4 presents a benchmarking analysis of the primary methods of Boon, with the objective of quantifying the time required for the application of each method to random Boolean networks. In conclusion (Section 5) we compare **BooN** to software distributions related to Boolean network analysis.

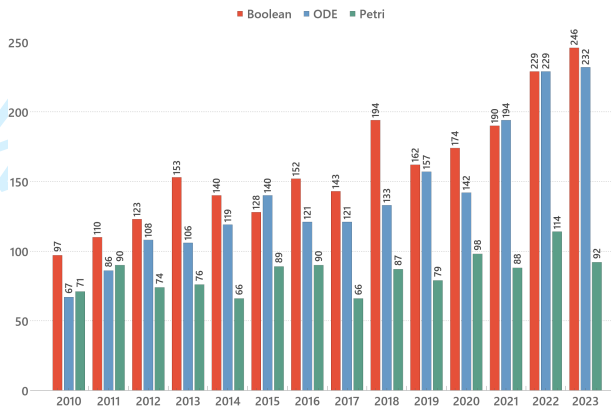


Fig. 1: Number of publications per modeling framework (Boolean network, ODE, Petri Net) on PUBMED.

2. Boolean Network

2.1. Basic definitions

A *Boolean network* is a discrete dynamical system operating on Boolean variables X . It is defined as a system of Boolean equations of the form: $x_i = f_i(x_1, \dots, x_n), 1 \leq i \leq n$ where each f_i is a logical propositional formula. We present in Figure 2 the dynamics of the following Boolean network:

$$F = \begin{cases} x_1 = (x_1 \vee x_2) \\ x_2 = (\neg x_3 \wedge x_1) \\ x_3 = (x_2 \wedge x_1) \end{cases} \quad (1)$$

A Boolean *state* of s is an interpretation of the variables of X in Boolean *i.e.*, $s : X \rightarrow \mathbb{B}, \mathbb{B} = \{0, 1\}$, and $S_X = (X \rightarrow \mathbb{B})$ is the set of all states.

The analysis of a Boolean network is primarily focused on the dynamics to investigate its behavior. In contrast to simulations conducted within other modeling frameworks, such as ODEs, the Boolean dynamics provide the full description of all potential trajectories, thereby ensuring a comprehensive and thorough investigation of all possible dynamical scenarios.

Formally, a *model of dynamics* is defined by a labeled transition system where for each transition the states of agents are updated with regard to an updating policy defined by a *mode*. Usually, two modes are used: the *asynchronous* mode where one variable exactly is updated per transition and the *synchronous* mode where all the variables are updated per transition. As example, the *asynchronous* mode leads to the following labeled transition system $\langle \rightarrow, X, \mathbb{B}^n \rangle$ where the updated agent $\rightarrow \subseteq S_X \times X \times S_X$ labels the transition relation, $\xrightarrow{x_i}$ such that:

$$s_1 \xrightarrow{x_i} s_2 \stackrel{\text{def}}{=} s_1 \neq s_2 \wedge s_2(x_i) = f_i(s_1) \wedge \forall x_j \in X \setminus \{x_i\} : s_2(x_j) = s_1(x_j).$$

A key objective of the analysis is to assess the equilibrium of the modeled network to gain insight into its long-term behavior. Basically, a state s is an *equilibrium* for $\rightarrow$, if it can be infinitely reached once met, *i.e.*, $\forall s' \in S_X : s \rightarrow^* s' \implies s' \rightarrow^* s$. The aim is to identify and characterize the attractors, which are defined as sets of mutually reachable equilibria. The attractors corresponding to a single state are called *stable states* and can be efficiently and quickly computed by symbolic method while the computation of the other attractors (cyclic) remains exponential in general, limiting their investigation to small networks.

Finally, an *interaction graph* $\langle X, \rightarrow \rangle$ portrays the causal interactions between variables of a Boolean network. An interaction $x_i \rightarrow x_j$ exists if and only if x_i occurs as literal in a minimal DNF form of f_j , *i.e.*, $x_i \rightarrow x_j \stackrel{\text{def}}{=} x_i \in V(\text{DNF}(f_j))$. The interactions are refined by a sign stipulating the nature of the interaction: positive (+) or negative (-). Figure 2 illustrates the asynchronous dynamics of Boolean network (1) leading to a cyclic attractor (red) and a stable state (yellow), and its signed interaction graph.

2.2. Controllability

Recently, a significant body of research has been devoted to the study of controllability in Boolean networks. The aim of this research is to identify the necessary modifications to the equations governing the network to figure out the means of satisfying a query at a given equilibrium state. The theoretical framework, and the detail of the computational methods developed in **BooN** for the controllability analysis can be found in [5] and [6] for its extension to control sequence.

Adding control extends the Boolean network to *Boolean control network* (BCN) where the *control parameters* u_i are added to the equation for figuring the control action. The classical modifications of equations consist in freezing a variable by setting it to True/1 or False/0. Basically, adding control extends the formula to include the control parameters. A freeze occurs when the control parameters freeze ($u_i = 0$) otherwise it is inactive ($u_i = 1$). The formalization of the control extension can be achieved as follows

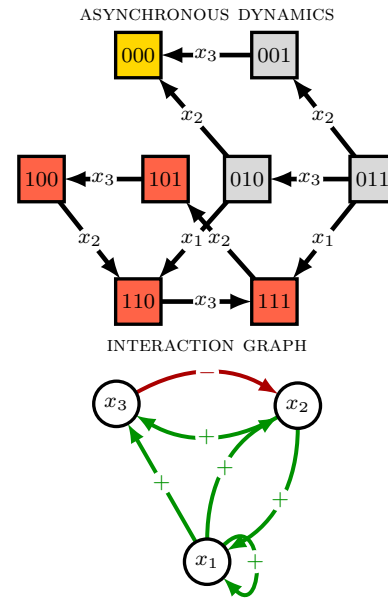


Fig. 2: Asynchronous dynamics and equilibria (in different colors for each attractor), interaction graph

for each equation:

$$x_i = f_i(x_1, \dots, x_n) \wedge u_i^0 \quad \text{for freezing to 0}$$

$$x_i = f_i(x_1, \dots, x_n) \vee \neg u_i^1 \quad \text{for freezing to 1.}$$

Note that when $u_i^0 = 0$ then the equation becomes $x_i = 0$ and similarly when $u_i^1 = 0$, $x_i = 1$.

The Boolean network (1), subjected to control thus becomes:

$$\begin{cases} x_1 = (x_1 \vee x_2) \wedge u_1^0 \vee \neg u_1^1 \\ x_2 = (\neg x_3 \wedge x_1) \wedge u_2^0 \vee \neg u_2^1 \\ x_3 = (x_2 \wedge x_1) \wedge u_3^0 \vee \neg u_3^1 \end{cases} \quad (2)$$

The BCN has now 6 control parameters. With this method, the control is applied on nodes because the variable is set to a Boolean value when the control freezes. The control can be also more selectively applied on links [5]. As the Boolean network is modified when the control freezes, the dynamics is altered leading to different equilibria.

The resolution of control seeks to identify the active control required to fulfill a specified query. This query articulates a condition related to stable equilibria, which may take the following forms: *possibility*, signifying the existence of at least one stable state that satisfies the query, or *necessity*, mandating that all stable equilibria must conform to the query. The two forms of queries are included in **BooN**'s controllability analysis.

To illustrate this situation, if the aim is to reach $x_3 = 1$ at a stable state, the sole control that will achieve this outcome is to activate u_2^1 *i.e.*, $u_2^1 = 0$ while $u_i^c = 1$ for the other configurations, the resulting BCN and its dynamics are shown in Figure 3. Now only one new stable state exists (111).

The primary challenge is to accurately determine the control parameters' freeze actions to regulate the dynamics to achieve the desired equilibrium. As previously referenced, a variety of methods have been developed to address this issue.

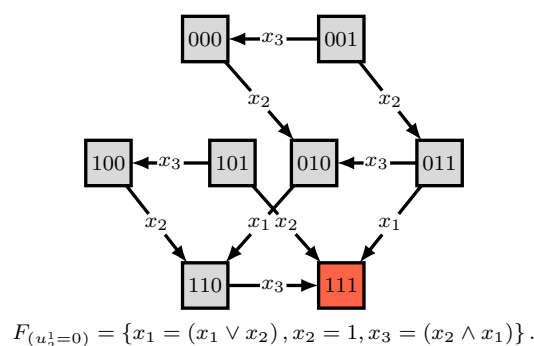


Fig. 3: Controlled network of F by assigning $u_2^1 = 0$.

Controllability analysis is a valuable investigative tool in drug target discovery and disease etiology. Typically, a biomarker profile identifies a specific state, either disease or healthy. From control analysis, we can determine which variables should be turned ON or OFF, that is frozen, in order to achieve the desired binary biomarker profile at stable state. This allows us to properly identify the actions that should be applied to genes, either over-expression or inhibition. In the distribution (see GitHub site), we have fully described an example by a Jupyter program illustrating the use of controllability analysis for the identification of the causes of diseases and the therapeutic target discovery applied to breast cancer. This analysis is also described in the README for its interactive application.

3. BooN

BooN facilitates the analysis of the Boolean network including the network design, the equilibrium computation and the controllability analysis. It is freely available in open source under GitHub [1]. **BooN** has also been added in Colomoto [7] repository collecting software related to Biological network analysis. The distribution includes the sources, complete documentation of the different methods applied to **BooN** object, tutorials and various examples including a real application of the controllability analysis on Breast cancer leading to drug target prediction. **BooN** was developed in Python and composed of 3 packages:

- **boon** is the core library providing a comprehensive set of methods for manipulating the **BooN** object.
- **logic** is a sub-library of **boon** containing the basic functions related to propositional logic: basic functions, formula pretty printing, CNF conversion, and prime implicants computation.
- **boonify** is the user-friendly graphical interface (GUI) interfacing the **boon** function based on PyQT5.

BooN has an internal file format for the storage of Boolean networks. However, for interoperability with other applications, Boolean networks in BoolNet, SBML, or Python-based format can also be imported.

Figure 4 illustrates the full range of analysis features accessible via different windows using the GUI for Boolean network (1).

Concerning the GUI, the design of the Boolean network is completely achieved from the drawing of the interaction graph (Figure 4). From the interaction graph drawing a DNF formula is generated. Therefore there is no conceptual limitation on the formulas since all formulas can be converted to DNF. The user

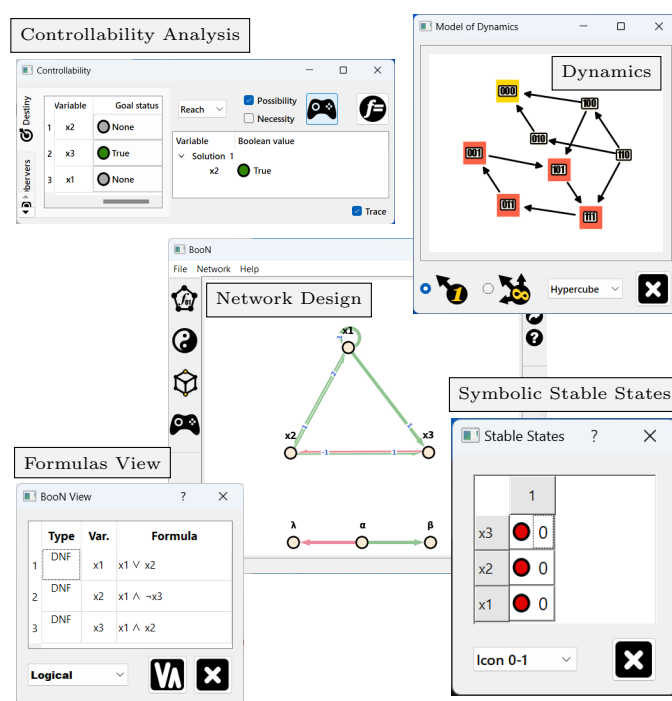


Fig. 4: Overview of the **BooN** functionalities/windows using GUI.

draws the positive or the negative interactions and specifies in which module the variable belongs. A module characterizes a set of cooperative variables. It is identified by an integer id and translated into a AND-clause in the DNF formula. Also note that the formula can be modified directly using the Formula View.

Moreover, the different analysis (Dynamics w.r.t. a mode, Controllability, symbolic Stable state computation) are accessible via a menu and buttons located in the both sides of the main window where the interaction graph is drawn. Comprehensive online help is also available.

More technically, the dynamics refers to calculating transitions between states in a mode that defines the state updating policy. The full dynamics computation is available in **BooN** in both the synchronous and asynchronous modes, and more generally for any possible mode. The symbolic computation of stable states efficiently lists the interpretations of the following equivalence stating the fixed point of the dynamics: $f(x) \iff x$. The resolution is achieved by a SAT solver, specifically the z3-Solver [8].

Moreover, the controllability analysis entails the computation of prime implicants, as detailed in the reference [5]. These prime implicants are used to extract the active control parameters which define the control to be applied.

Given that a control is active if it is equal to 0 ($u_i^c = 0$), we extract the negative control parameters from the prime implicants, which are true only when they are set to 0, for the purpose of defining the core, which is the minimal set of active control parameters under the inclusion. The other variables, positive controls and network variables, are discarded since they do not share information related to the control activity. The prime implicant algorithm that has been implemented is based on the translation of the problem into an integer linear programming problem [9] and the integer linear programming solver used is Pulp [10].

4. Benchmark

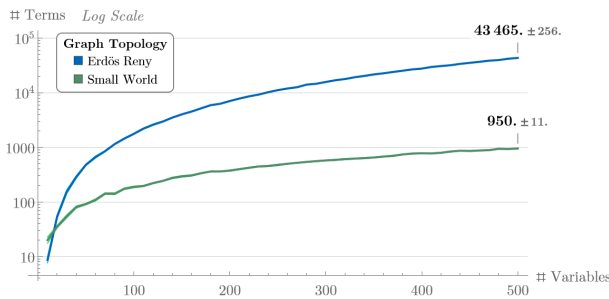


Fig. 5: Number of terms per network size and topology.

The primary methods of **BooN** are bench-marked using randomly generated Boolean networks. We classified these networks into two categories based on the topology of their interaction graphs: Erdős-Rényi and Small-World. The formulas were constructed from interaction graphs defining variable dependencies. For each variable, a disjunctive normal form (DNF) formula is randomly generated from the input neighbors of the variable within the interaction graph. The number of clauses is randomly defined within predefined bounds, and the negations are determined from a parameter controlling the likelihood of positive terms. Formulas derived from the Erdős-Rényi interaction graph proved more complex, containing a greater number of terms than those from the Small-World interaction graph (Figure 5). Consequently, the benchmark analysis was divided into two formula categories: complex formulas (Erdős-Rényi) and simpler formulas (Small-World). The benchmarks were computed on a INTEL Icore 7 (1.90 Ghz) with 16Go of RAM.

The benchmark assesses the efficiency of the methods on networks of increasing size in terms of the number of variables. For each size, 10 networks are randomly generated. The curves show the average for each size with its uncertainty. The Bench-marking Jupyter program is included in the GitHub distribution. We respectively assess the stable state computation, the brute force equilibria computation and the controllability analysis.

4.1. Stable states

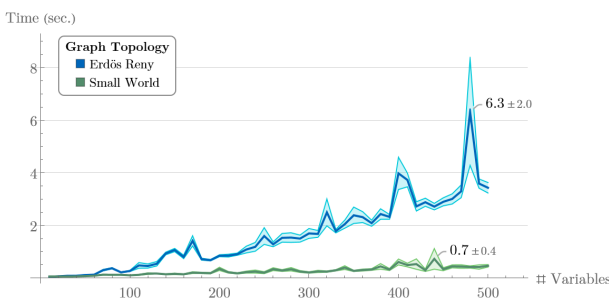
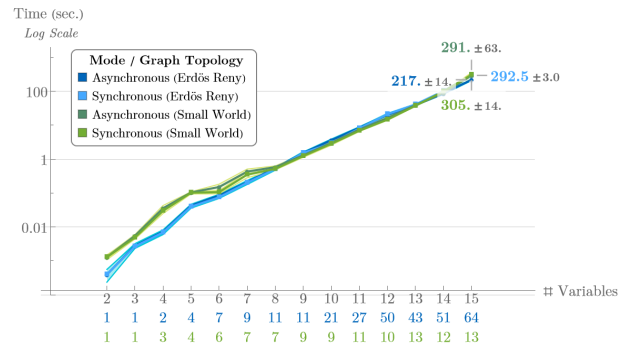


Fig. 6: Time for stable states computation.

The stable state method based on symbolic resolution using the Z3 SAT solver is a very fast and scalable approach, with processing times of less than 6.3 sec for large networks comprising thousands of terms (Figure 5). The complexity of the formulas has a significant impact of the time computation.

4.2. Equilibria

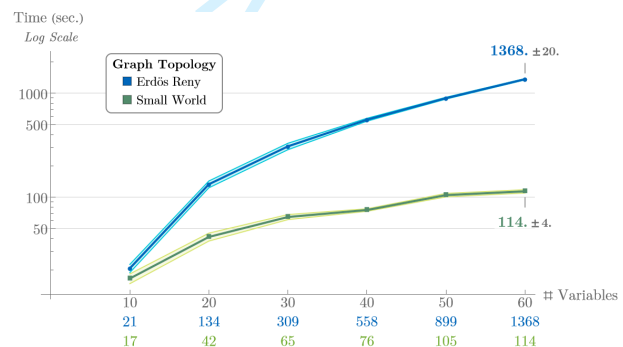


Below the number of variables is the number of terms for the Erdos-Rényi and Small-World networks.

Fig. 7: Time for equilibria computation.

The brute force algorithm characterizing the attractors is performed on the whole state space ($2^{|X|}$ states). As might be expected, the time required is doubled with each additional variable, and thus follows an exponential growth curve. Nevertheless, the algorithm has been optimized to remain efficient when considering a limited number of variables, which in this case is 15. Our analysis revealed no notable discrepancies between the Small-World and Erdős-Rényi formulas. This is because the primary constraint stems from the exponential number of states, rather than the complexity of the formulas themselves.

4.3. Controllability



Below the number of variables is the number of terms for the Erdős-Rényi and Small-World networks.

Fig. 8: Time evaluation for controllability (possibility) queries.

The final benchmark pertains to the evaluation of controllability. The queries ascertain some requisite conditions corresponding to a Boolean profile for the variables x_0 and x_1 . The selected profile

is never satisfied in the current stable states. The goal is to possibly reach this profile at stable state (in 1 stable state at least). The benchmark demonstrates that computing the control is efficient for networks with a significant number of nodes. However, the performance is contingent on the complexity of the formulas, thereby explaining the observed time difference between Erdős-Renyi and small-world networks. Indeed, the number of terms has a direct influence on the resolution capability of the PULP ILP-solver, since the system to be solved will be more complex.

5. Conclusion

Several software packages are also available for Boolean network analysis. These are written in different languages and offer a range of functions for performing this analysis.

GINSIM [11] provides an interactive design of the Boolean network, including annotations using a GUI. It offers a step-wise simulation and full dynamics for any mode, as well as a computation of stable states based on symbolic analysis and a full description of attractors from a brute force algorithm limiting its application to small network. It has been developed in JAVA.

BoolNet [12] is a R-package also devoted to Boolean network analysis. It tackles with the synchronous and asynchronous analysis and also probabilistic Boolean network. It includes functions for various simulations, attractor search, and robustness analysis but does not contain symbolic algorithm for stable states computation. The specification of a Boolean network is achieved from formula description. BoolNet is reduced to a package and does not contain a graphical interface for facilitating the network design. Similarly PyBoolnet [13] is a Python package devoted to Boolean network analysis including some graph facilities. The Python package AON [14] is another tool devoted to the analysis of asynchronous dynamics. It allows the examination of partially specified Boolean networks with uncertain update functions. The package is based on the binary decision diagram, which enhances the attractor detection algorithms.

BoolSi [15] is an open-source cross-platform command line tool restricted to simulations of synchronous Boolean networks. It supports parallel processing using MPI. BoolSi is used to model the behavior of complex dynamic networks and allows for identification and statistical analysis of network attractors.

BooN is similar to GINSIM in terms of its functionalities. In comparison to the BoolNet package, Boolean Networks is devoted to pure Boolean networks. However, it provides a package and a graphical interface which provides a convenient way to visualize and interact with the network. It also offers unique capabilities for controllability analysis, which is not available in other software.

A number of software applications are devoted to controllability analysis, as CABEAN [16] or Bonesis [17]. However, they are focused on this specific analysis and do not encompass the other aspects related to Boolean dynamics analysis addressed by the aforementioned software.

In conclusion, BooN is a software solution tailored for Boolean network dynamics analysis, featuring a unique ability to conduct controllability analysis. Furthermore, its graphical interface allows for the straightforward manipulation of Boolean networks.

References

1. Franck Delaplace. Franck-Delaplace/BooN, September 2024.
2. Stuart Kauffman. Homeostasis and differentiation in random genetic control networks. *Nature*, 224:177–178, 1969.

3. Leon Glass and Stuart A Kauffman. The logical analysis of continuous, non-linear biochemical control networks. *Journal of Theoretical Biology*, 39(1):103–129, 1973.
4. René Thomas. Boolean formalization of genetic control circuits. *Journal of Theoretical Biology*, 42(3):563–585, 1973.
5. Célia Biane and Franck Delaplace. Causal Reasoning on Boolean Control Networks Based on Abduction: Theory and Application to Cancer Drug Discovery. *IEEE/ACM Transactions on Computational Biology and Bioinformatics*, 16(5):1574–1585, September 2019.
6. Jérémie Pardo, Sergiu Ivanov, and Franck Delaplace. Sequential reprogramming of biological network fate. *Theoretical Computer Science*, 872:97–116, June 2021.
7. Aurélien Naldi, Céline Hernandez, Nicolas Levy, Gautier Stoll, Pedro T. Monteiro, Claudine Chaouiya, Tomáš Helikar, Andrei Zinovyev, Laurence Calzone, Sarah Cohen-Boulakia, Denis Thieffry, and Loïc Paulevé. The CoLoMoTo Interactive Notebook: Accessible and Reproducible Computational Analyses for Qualitative Biological Networks. *Frontiers in Physiology*, 9, June 2018.
8. Leonardo de Moura and Nikolaj Bjørner. Z3: An Efficient SMT Solver. In C. R. Ramakrishnan and Jakob Rehof, editors, *Tools and Algorithms for the Construction and Analysis of Systems*, pages 337–340, Berlin, Heidelberg, 2008. Springer.
9. C. Pizzuti. Computing prime implicants by integer programming. In *Proceedings Eighth IEEE International Conference on Tools with Artificial Intelligence*, pages 332–336, 1996.
10. Stuart Mitchell, Michael O’Sullivan, and Iain Dunning. PuLP: A Linear Programming Toolkit for Python.
11. A. Naldi, D. Berenguier, A. Fauré, F. Lopez, D. Thieffry, and C. Chaouiya. Logical modelling of regulatory networks with GINsim 2.3. *Bio Systems*, 97(2):134–139, August 2009.
12. Christoph Müssel, Martin Hopfensitz, and Hans A. Kestler. BoolNet—an R package for generation, reconstruction and analysis of Boolean networks. *Bioinformatics*, 26(10):1378–1380, May 2010.
13. Hannes Klärner, Adam Streck, and Heike Siebert. PyBoolNet: A python package for the generation, analysis and visualization of boolean networks. *Bioinformatics*, 33(5):770–772, March 2017.
14. Nikola Beneš, Luboš Brim, Ondřej Huvar, Samuel Pastva, David Šafránek, and Eva Šmířáková. AEON.py: Python library for attractor analysis in asynchronous Boolean networks. *Bioinformatics*, 38(21):4978–4980, November 2022.
15. Vladyslav Oles and Anton Kukushkin. BoolSi: A Tool for Distributed Simulations and Analysis of Boolean Networks. *Journal of Open Research Software*, 8(1), October 2020.
16. Cui Su and Jun Pang. CABEAN: A software for the control of asynchronous Boolean networks. *Bioinformatics*, 37(6):879–881, March 2021.
17. Stéphanie Chevalier, Déborah Boyenval, Gustavo Magaña-López, Théo Roncalli, Athénaïs Vaginay, and Loïc Paulevé. BoNesis: A Python-Based Declarative Environment for the Verification, Reprogramming, and Synthesis of Most Permissive Boolean Networks. In Roberta Gori, Paolo Milazzo, and Mirco Tribastone, editors, *Computational Methods in Systems Biology*, pages 71–79, Cham, 2024. Springer Nature Switzerland.

ANSWER TO REVIEWERS.

I would like to address a sincere word of thanks to the anonymous reviewers for their constructive and insightful comments to provide valuable feedback on our article. I paid heed to the questions, pieces of advice and suggestions made by the reviewers and have accordingly modified the manuscript.

To ease the reading of the responses, I have adopted the following color convention: in black the review and in blue my response.

Reviewer: 1

General remarks

In this paper, the author introduces BooN, an open-source Python-based software designed for Boolean network modelling. The key contribution is the comprehensive integration of tools for Boolean network design, dynamics modelling, symbolic computation of stable states, and controllability analysis. In addition, all these functions are available via a graphical user interface, supported by a tutorial on how to use it. In this way, the method provides a facility offering both user-friendly and technically robust functionality, addressing the needs of both beginners and advanced users in biological network modelling.

The method's key contribution is its controllability analysis, which is crucial for studying gene regulatory networks in pathologies or drug target discovery. Moreover, its integration into the Colomoto framework enhances its usability and interoperability with other network analysis tools. Finally, the tool supports multiple input formats, making it adaptable for researchers working with different data formats.

I will list below some points for improvement to further strengthen its impact:

While the author mentions the availability of documentation and examples, more detail on real-world case studies or an extended example (beyond breast cancer) could enhance understanding of how BooN performs in practice.

We have completed the jupyter code [breast-cancer-study.ipynb](#) related to breast cancer study, by adding comments concerning the explanation of the analysis, and also for the example. I hope this complement will be sufficient.

The article could be more impactful if performance comparison is provided, especially on large-scale networks. While the author highlights the tool's limitations for analysing larger cyclic

attractors, explicit details on computational efficiency compared to other tools would give users a clearer sense of BooN’s scalability.

I have added a section detailing the benchmarks of the main method of BooN.

The software currently supports Boolean control networks with node-level interventions. Future work could explore extending control options to more refined network components, such as edges, which would add flexibility for modelling complex biological interactions.

Effectively the method is restricted to nodes. However, we had already developed the computational framework to control edges. We do not include it because the control on edge does not seem to be practically used for biological application. For example, in drug discovery the issue is to identify a gene, and not its interaction. However, thanks to you comment we plan to extend the controllability on edges in the future, since you think it can be useful.

Concerning the GitHub repository:

The jupyter notebooks breast-cancer-study.ipynb and example.ipynb need more explanations on each step of the analysis. In addition, in these examples, building the interaction graph is done near the end of the analysis; I suggest this part be moved just after importing or introducing the regulatory network.

We have completed the jupyter code breast-cancer-study.ipynb related to breast cancer study, by adding comments concerning the explanation of the analysis, and also for the example. We also move the interaction graph in the network analysis at the beginning of the program. .

Grammar suggestions/corrections:

We have applied for the correction. Thank you for your suggestions.

Adding the hyperlink to the GitHub repository would facilitate the tool’s accessibility.

I have added a link to access to the site directly.

Reviewer: 2

Comments to the Author

[Review] BooN: Boolean Network Analysis Software

The application note presents the BooN toolbox, which supports manipulation of Boolean networks, symbolic computation of fixed-points, explicit computation of complex attractors, and identification of permanent control interventions. According to the author, the main novel contributions of BooN are (broadly speaking) its controllability method and its graphical user

1
2
3 interface. Overall, these are not exceptionally novel (e.g. other controllability/GUI tools exist),
4 but I think BooN is a welcome addition into the ecosystem of Boolean network tooling and adds
5 to the diversity of available methods and formats. However, I have some reservations regarding
6 the overall presentation and quality of the tool that I think are easily addressable and would
7 make this a much more interesting submission.
8
9

10 11 **Comments regarding text**

12
13 Overall, I think the text spends a lot of time explaining what Boolean networks are and how
14 they work, but not that much is actually said about BooN itself. The BooN section + Conclusion +
15 Fig. 4 are only one page of the whole text, the rest is introduction and theory. I understand that
16 the work needs to be properly motivated and important concepts must be explained. But with a
17 5 page limit, I believe that at least some of the theory can be perhaps simplified or moved to a
18 supplementary material to provide more room for talking about BooN.
19
20
21

22 Instead, what I would suggest:

23
24 * There is no mention of performance or benchmarks anywhere in the text. Even if a proper
25 benchmark is not available, at least some discussion of BooN's capabilities in that regard would
26 be welcome.
27
28

29
30 We have added a section for the BooN benchmark evaluating the primary methods of BooN:
31 stable state computation, brute force equilibria computation and controllability analysis.
32

33
34 * Maybe some extra discussion can be dedicated to the control method. I understand that it is
35 already published, but a more comprehensive summary would be interesting.
36

37 The control analysis is fully detailed in [Biane2019]. Essentially, we summarize the approach. In
38 response to your comment, I have tried to add some elements explaining the controllability,
39 notably its application in the article, but I do not really know what I can add which is not already
40 published concerning the method. Moreover, I have added a sentence to refer to this article for
41 the theoretical background study.
42
43

44
45 For example, you mention that the method identifies negative control parameters, but what
46 about positive control parameters?
47
48
49
50
51
52
53
54
55
56
57
58
59
60

1
2
3
4
5
6
7
8
9
10
11
12
13
14
15
16
17
18
19
20
21
22
23
24
25
26
27
28
29
30
31
32
33
34
35
36
37
38
39
40
41
42
43
44
45
46
47
48
49
50
51
52
53
54
55
56
57
58
59
60

Concerning the negative control parameters, the control is active when its value = 0. This is a convention motivated by the fact that the control freezes the variable and 0° is the freeze temperature ;-).

Therefore, since we need to define which control parameters are active, (ie. equal to 0) we need to extract negative control parameters in the result of the prime implicant computation which is a minterm because these terms are true only when the control is false. Moreover, the positive control parameters thus correspond to inactive control, and thereby they are not considered since we focus on the active control.

I have given more explanation in the article to explain this element.

* Perhaps more could be also said about what is available in the boon and logic Python modules? You mentioned that the Boolean network is completely defined by the interaction graph. How does this work for complex, high arity functions?

The conversion of the interaction graph with the specification of modules (integer identifier) leads to a DNF formula. There is no limitation of size and all types of formulas can be encoded since it was a DNF except the graphical limitation for drawing very large graph. We have completed the explanation in the article.

Are there some functions that you cannot represent in this way? Do you use some existing libraries to provide the SBML support?

BooN is able to read SBML file but not not generate them (yet). We use the standard SBML package libsbml.

* Another option could be a small biological case study that illustrates the usefulness of controllability analysis and the fact that BooN can be applied to realistic biological models. Maybe even implementing the example network (i.e. equations (1) and (2)) in BooN and showing the result would be interesting.

The usefulness of the controllability analysis is fully illustrated in the example breast-cancer study.py. This example was also fully described in our article which is dedicated to controllability analysis previously mentionned.

A sentence was added to the article to direct readers interested in concrete applications of controllability to the example included in the distribution.

* (However, do note that these are only suggestions, I don't think it is necessary to add all of this)

* The references and related work could be better:

* As far as I can tell, application notes have a limit of 10 references, this text has 20. However, this limit is often exceeded in other application notes that I found published in this journal, so I don't consider this to be a major problem.

* With that said, references 5-12 (except 7) are only used to "generally" introduce the controllability problem, but are (as far as I can tell) not directly related to methods implemented by BoonN, or not directly compared to BoonN. I'm by no means saying these are irrelevant, but with a limit like this, I would expect some more explicit connection to the tool itself.

We have removed the citations you have mentioned mainly used to introduce the controllability but which are not discussed and compared. A full state of art comparison of the method and those mentioned here was achieved in the article [Biane2019].

* Meanwhile, BoolNet is not cited properly, even though it has an associated publication [Mussel2010], and other tools from the Colomoto docker that are similar in terms of capabilities are also not mentioned.

The reference is added..

For example, pyboolnet [Klarner2017] also supports Boolean network manipulation, computation of attractors, and prime implicants (it does not support control as far as I know). Also AEON.py [Benes2022] is another library for Boolean network manipulation, symbolic attractor/steady state computation and control (I believe it also has a web GUI?). Both are available in Python.

In accordance with your request, I have incorporated the two references that you specified. I am grateful to you for your comprehensive and interesting review. Concerning AON I do not see any reference to a Graphical interface in the documentation.

Other minor issues

* The first reference would really benefit from having a URL. I can find the repository based on its name, but it is somewhat tedious.

I have added an hyperlink

* The sentence on lines 8-10 on page 2 does not make much sense. Seems like some content is missing?

I have proposed an alternative sentence. I hope, this will be clearer. Otherwise I suggest to remove it since it is not central.

* S_X (line 22, page 2) is not defined anywhere, but considering what I said about simplifying the theory section, I would just replace it with $\mathbb{B}^n$?

We define S_X which is the set of all logical interpretations corresponding to functions assigning Boolean value to variables. Now, I have defined S_X before.

* Listing the exact queries in Fig. 1 also seem rather unnecessary for this kind of paper. Personally, I would move this to a supplement.

We remove them.

* Page 2, line 34: To me, "attractors reduced to a single state" implies that the attractors are somehow modified (by a reduction) to obtain one state. Maybe "corresponding to a single state" would be better

We rephrase the sentence according to your suggestion

* I believe it's written i.e., not ie.?

Actually it was a mistake in my \ie command, I correct it. thank you.

Comments regarding tool

* I was able to install the tool and run the example/case study without any issues. I especially appreciate the provided documentation. I was also able to load a somewhat larger network (285 variables) and compute its stable states.

* With that said... the readme and documentation don't say how to start the GUI? I eventually figured it out, but for new users, this is probably something you want to explain.

I have corrected this in the README

* The GUI works... but I have had some issues with usability.

* If I press enter repeatedly while a node or edge isn't selected (i.e. as if adding label to a node), the app prints an error into the command line.

Sometimes it recovers without issues, but it seems that depending on other actions I performed (or other app windows I have open?), this can break the option to edit update functions or to kill/freeze the app outright.

Thank you for completely testing the application. Actually, the graphical interface is based on EditableGraph of NetGraph. The acquisition of a BooN is achieved by catching the events of EditableGraph. Effectively in this function I have also remarked that the edition is frozen by entering Return several times without pointing to a node. Hence, I cannot correct it by my own unless by fully defining a new version for acquiring network graphically, which is planned for 2025. However, I have already send a message to the developer of Netgraph concerning this issue.

* Sometimes if I try to edit an update function but don't actually change anything, the app will report a syntax error. One time, it even started repeating this error dialog after I closed it and had to be killed.

For the modification of the formula using the formula view you first need to shift from logical to Python because only the python syntax is parsed. I have now added this tip in the syntax error message to guide the user.

* Is there some sort of loading indicator for computing the steady states and attractors when a complex model is analyzed? I have not encountered one.

The stable state computation is very fast and based on a Z3 SAT Solver and I do not know if there is such indicator in the solver. For the attractor computation, this can be added and it will be an issue.

[Mussel2010] Müssel, Christoph, Martin Hopfensitz, and Hans A. Kestler. "BoolNet—an R package for generation, reconstruction and analysis of Boolean networks." *Bioinformatics* 26.10 (2010): 1378-1380.

[Klärner2017] Klärner, Hannes, Adam Streck, and Heike Siebert. "PyBoolNet: a python package for the generation, analysis and visualization of boolean networks." *Bioinformatics* 33.5 (2017): 770-772.

[Benes2022] Beneš, Nikola, et al. "AEON. py: Python library for attractor analysis in asynchronous Boolean networks." *Bioinformatics* 38.21 (2022): 4978-4980.

Reviewer: 3

Comments to the Author

1. General assessment of the work:

in this work, Delaplace et al. report the development of a Boolean Network analysis package called BooN. Overall the package seems to be an interesting proposition for the use of Boolean network models in biology (and other fields).

1

2

3

4

5

6

7

8

9

10

11

12

13

14

15

16

17

18

19

20

21

22

23

24

25

26

27

28

29

30

31

32

33

34

35

36

37

38

39

40

41

42

43

44

45

46

47

48

49

50

51

52

53

54

55

56

57

58

59

60

2. Specific comments for revision (mandatory):

- Although the software looks useful, I do not see what the software does that goes beyond what is already in the field. Is the novelty the ability to do the controllability analysis? Or is it the visualization?

- What is it that BooN does that others do not?

Mainly, BooN the novelty is the integration of the controllability analysis in a platform dedicated to Boolean network analysis. Moreover we can perform this analysis using the graphical interface. The other features are also in the other software related to Boolean network analysis. Moreover, as far as I know, there is no such development in python including a graphical interface for Boolean network manipulation. Therefore, BooN offer thus this development for the scientific community since the sources are provided in the distribution.

- The figure showing the number of publications per modeling framework further highlights the need to show why BooN is an improvement over all of these.

- The author mentions that Boolean evaluation can be synchronous or asynchronous. Can BooN do both?

Yes it can do both. I have added a sentence to insist on this fact.

- What does the controllability analysis enable in a bioinformatics context?

The controllability analysis is central for discovering the molecular causes of the diseases and predict drug targets. Mainly the controllability identifies the variable that must be frozen to True or False in order to reach an expected fate. The expected fate usually depicts a biomarker profile. Therefore the controllability analysis could answer to the following question: Given a Boolean network which genes must be frozen to cure a disease identified by a healthy biomark profiles.

- Figure 4 would be better as a workflow.

There is no real workflow but rather several services. I am sorry bu I do not understand by workflow here.

- How does one use BooN? A tutorial for use would be useful either with the paper and/or on the GitHub site.

A tutorial is available in the GitHub site (Example/example.py) showing the main features of Boon. Moreover, in README I also detail an interactive application of Boon using the GUI on Breast cancer study.

3. Specific comments for revision (optional):

- Figure 1 is uninformative.

I'm sincerely sorry I don't understand your comment. To what extent is the figure showing the number of publications in Pubmed with associated queries not informative?

- The conclusion reads more like an introduction, maybe start with that ?

Yes you are right it could. However, in conclusion we have also included a comparison with the current software, and we think it would be better to achieve this comparison after explaining Boon.

- The paper switches from high-level, to very specific maths, to high level in a way that makes it hard to follow. Given the broad readership of the journal, the authors may consider a supplement for the detailed info.

The controllability analysis is detailed in [Biane2109] I have added a sentence to invite the reader to refer to this article for more details.

- Visual Boon seems to be a really interesting tool. Could you tell us more about the tool and what the visualization does?

In the article, I have incorporated additional details regarding the graphical interfaces.

[Biane2109] Célia Biane, Franck Delaplace - Causal reasoning on Boolean control networks based on abduction: theory and application to Cancer drug discovery. IEEE/ACM Trans Comput Biol Bioinform. 2018 Dec 21.